

Индексы по выражениям

Индекс можно создать не только по полю таблицы, но и по функции или скалярному выражению с одним или несколькими полями таблицы. Это позволяет быстро находить данные в таблице по результатам вычислений.

Выражение, на основе которого будет сформировано поле индекса, описывается при определении полей индекса (Index fields), опцией Expression:

The screenshot shows the 'Index element' dialog box with the following fields and values:

- Kind:** (empty)
- Name:** (empty)
- Expression:** `COALESCE("ДатаЗакрытия", 'infinity')`
- Order:** `Ascending`
- Operator class:** (empty)
- Null location:** `Default`
- Collation:** (empty)

Пример 1: для сравнений без учёта регистра символов часто используется функция `lower`.

Индекс, определенный для результата функции `lower(col)`, может быть описан так:

Index element

Kind ⓘ
Name ⓘ
Expression ⓘ

Expression

Expr_0

1

lower("Фамилия")

Order ⓘ
Operator class ⓘ
Null location ⓘ
Collation ⓘ

Ascending
Default

 Если объявить этот индекс [уникальным \(is unique\)](#), он не даст добавить строки, в которых значения col (поля "Фамилия", "Имя", "Отчество") различаются только регистром, как и те, в которых значения col действительно одинаковые. Таким образом, индексы по выражениям можно использовать ещё и для обеспечения ограничений, которые нельзя записать как простые ограничения уникальности.

Пример 2: Индекс будет определен как конкатенация (соединение) полей таблицы "Фамилия", "Имя" и "Отчество", для поиска без учета регистра:

Expression

1

lower("Фамилия" || ' ' || "Имя" || coalesce(' ' || "Отчество", ''))

Пример 3: Определение индекса для текстового поиска по фамилии, имени и отчеству сотрудника:

Expression

1

to_tsvector('simple'::regconfig, "Фамилия" || coalesce(' ' || "Имя" || coalesce(' ' || "Отчество", '')))

Пример 4: Использование условного выражения:

Expression

1

case

2

when("Тип" = 30) then

3

"Сервер"

4

else

5

null

6

end

Примеры используемых в индексах скалярных выражений:

```

1 -- индекс будет определен по числу ('месяц'*100 + 'день')
2 date_part('month', "ДатаРождения" )*100 + date_part('day', "ДатаРождения"

```

```

3 )
4 -- индекс будет определен для строк, в которых поле "Тип" нулевое или
   равно двум
5 "Тип" = 0 OR "Тип" = 2
6
7 -- индекс будет определен для строк с вторым элементом массива поля
   "Флаги" и пятым – поля "СтатусПартнера"
8 "Флаги"[2] AND "СтатусПартнера"[5]
9
10 -- индекс определен по числу, полученному извлечением подстроки –
   второго и третьего символа поля "НомерМашины"
11 substr("НомерМашины",2,3)
12
13 -- индекс определен по наибольшему значению из списка дат
14 greatest("ДатаОплаты", "ДатаОтгрузки")
15
16 -- индекс определен для строк, в которых значение поля "Категория"
   находится в диапазоне от 10 до 20
17 "Категория" BETWEEN 10 AND 20

```

Подробно синтаксис скалярных выражений изложен в статье ["Скалярные выражения"](#).

Поддержка индексов по выражениям обходится довольно дорого, так как эти выражения должны вычисляться при добавлении или изменении данных таблицы. Однако при поиске индексируемое выражение не вычисляется повторно, так как его результат уже сохранен в индексе.

В рассмотренных выше случаях система видит запрос как

```
WHERE поле_индекса = 'константа'
```

и поэтому поиск выполняется так же быстро, как и с простым индексом.

Таким образом, индексы по выражению могут быть полезны, когда скорость извлечения данных гораздо важнее скорости добавления и изменения.